

Initial Setup

Create a repository:

```
mkdir git-branching-demo
cd git-branching-demo

git init

echo "<h1>Home Page</h1>" > page1.html

git add .
git commit -m "Initial website"
```

Current state:

```
main
└─ Initial website
```

Demo 1: Fast-Forward Merge

Goal

See what happens when the target branch has not moved.

Steps

Create feature branch

```
git checkout -b feature-about-page
```

Add page2.html:

```
echo "<h1>About Us</h1>" > page2.html

git add .
git commit -m "Add About page"
```

Graph:

```
main
└─ A

feature-about-page
└─ A --- B
```

Switch back to main

```
git checkout main
```

Merge

```
git merge feature-about-page
```

Git output should mention:

```
Fast-forward
```

See history

```
git log --oneline --graph --all
```

Result:

```
A --- B (main)
```

Important Point

- No new merge commit created.
- Git simply moved main forward.
- Happens when main has no additional commits.

Demo 2: Recursive / 3-Way Merge

Goal

See merge commit creation.

Steps

Start from main

Create branch:

```
git checkout -b feature-contact-page
```

Add page3.html:

```
echo "<h1>Contact Us</h1>" > page3.html

git add .
git commit -m "Add Contact page"
```

Return to main

```
git checkout main
```

Make another change directly on main:

```
echo "<h1>Services</h1>" > page4.html

git add .
git commit -m "Add Services page"
```

Graph before merge:

```
graph TD
    A --- B
    B -.-> C
    C --> D["D (main)"]
    A --> D
```

The graph shows a sequence of commits. Commit A is the base. Commit B is a child of A. Commit C is a child of B, representing the 'feature-contact-page' branch. Commit D is the result of merging C back into the 'main' branch. The main branch is labeled 'D (main)'.

Merge

```
git merge feature-contact-page
```

See graph

```
git log --oneline --graph --all
```

Result:

```

      C
     /
A--B--D----M

```

(M = merge commit)

Important Point

- Both branches moved independently.
- Git must combine histories.
- New merge commit created.
- This is the classic 3-way merge.

Notes

1. Modern GIT shows **ort** instead of **recursive**.

ort stands for "**O**stensibly **R**ecursive's **T**win".

It is the newer merge strategy that replaced the older **recursive** strategy as the default in Git (starting with Git 2.34). Even though Git may See:

```
Merge made by the 'ort' strategy.
```

people still commonly refer to it as a **recursive merge** or **3-way merge**.

2. Why is it called a 3-way merge? Which three versions are used?

Git compares three snapshots:

```

      Feature Branch
          C
         /
A ----- B -----
         \
          D
      Main Branch

```

The three inputs are:

1. **Common ancestor (merge base)** → B
2. **Current branch tip (HEAD)** → D
3. **Branch being merged** → C

Git computes:

```
(B → D)  changes
(B → C)  changes
```

and combines them to produce the merge result.

So the "three ways" are:

- Common ancestor
- Current branch
- Incoming branch

That's why it's called a **3-way merge**.

3. What is a merge commit? Does it combine all feature branch commits into one commit?

A **merge commit** does not squash or combine feature branch commits into one commit.

Suppose you have:

```
A---B---D (main)
      \
      C---E (feature)
```

After merge:

```
A---B---D-----M (main)
      \         /
      C---E-----/
```

The new merge commit **M** contains:

- A snapshot of the files after combining both branches.
- **Two parent pointers:**
 - Parent 1 = D (main)
 - Parent 2 = E (feature)

The original commits **C** and **E** remain intact in history.

Important:

After the merge, Git history still Sees:

```
A
B
C
D
E
M
```

The merge commit **does not replace** C and E.

When do commits get combined into one?

Only when you use:

```
git merge --squash feature
```

or GitHub's "**Squash and Merge**" option.

Then:

```
A---B---D---S
```

where S is a single commit containing the net effect of all feature branch changes. The individual commits (C, E, etc.) are not brought into the main branch history.

4. Can I delete a branch after merge?

You can absolutely delete the feature branch after a recursive (3-way) merge.

Example:

```
A---B---D-----M  (main)
      \             /
      C---E-----/  (feature)
```

After the merge, commit M points back to both parents (D and E). Because of those parent links, commits C and E are now reachable through main.

So you can safely do:

```
git branch -d feature
```

Result:

```
A---B---D-----M (main)
      \         /
      C---E-----/
```

The branch name **feature** is gone, but the commits are still preserved in Git history through the merge commit.

What a branch really is?

A branch is just a pointer to a commit.

Before deletion:

```
feature -> E
main    -> M
```

After deletion:

```
main -> M
```

Deleting a branch doesn't delete the commits. It only deletes the pointer.

Commits **C** and **E** still exist because **M** references them indirectly.

This is why deleting **feature** branches after merging is a standard practice in most teams.

Demo 3: Rebase

Goal

See history rewriting and linear history.

Steps

Create branch

```
git checkout main
git checkout -b feature-gallery
```

Add file:

```
echo "<h1>Gallery</h1>" > page5.html

git add .
git commit -m "Add Gallery page"
```

Move main ahead

```
git checkout main

echo "<h1>FAQ</h1>" > page6.html

git add .
git commit -m "Add FAQ page"
```

Graph:

```
main
A---B---C
      \
      D (feature-gallery)
```

Rebase feature branch

```
git checkout feature-gallery
git rebase main
```

Git does not move the commits onto main.

Instead, it rewrites the commits of feature-gallery so they appear as if they were created from the latest main.

- Git follows these steps:
 1. Temporarily saves changes from D
 2. Moves feature-gallery pointer to C
 3. Replays D on top of C

Example:

Graph becomes:

```
A---B---C---D'
```

(D' is rewritten commit)

- Notice:
 1. 'D' is a new commit
 2. Commit hash changes

Merge back

```
git checkout main  
git merge feature-gallery
```

Output:

```
Fast-forward
```

Important Point

- Rebase = "take my branch commits and replay them on top of another branch."
- Rebase rewrites commit history.
- Creates a cleaner linear history.
- After rebase, merge becomes fast-forward.

Nice Comparison

See:

```
git log --oneline --graph --all
```

after merge commit approach and after rebase approach.

Students immediately see the difference.

Demo 4: Merge Conflict

Goal

Create a real conflict and resolve it.

Start

Use page1.html.

Current content:

```
<h1>Home Page</h1>
```

Create branch

```
git checkout main
git checkout -b feature-home-redesign
```

Modify:

```
echo "<h1>Welcome to Our Website</h1>" >> page1.html
```

Commit:

```
git add .
git commit -m "Redesign home page title"
```

Switch to main

```
git checkout main
```

Modify same page differently:

```
echo "<h1>Company Portal</h1>" >> page1.html
```

Commit:

```
git add .
git commit -m "Update home page title"
```

Graph:

```
      feature-home-redesign
      |
      C
A --- B ----- D (main)
```

Merge

```
git merge feature-home-redesign
```

Git says:

```
CONFLICT (content)
Automatic merge failed
```

See conflict markers

Open file:

```
<<<<<< HEAD
<h1>Company Portal</h1>
=====
<h1>Welcome to Our Website</h1>
>>>>>> feature-home-redesign
```

Resolve

Replace with:

```
<h1>Welcome to Company Portal</h1>
```

Complete merge

```
git add page1.html
git commit
```

See graph

```
git log --oneline --graph --all
```

Important Point

- Git does not know which version is correct.
- Human must decide.
- Conflict markers explain both sides.

Demo 5: GitHub Pull Request (Two People)

This is a real team workflow.

Setup

Create a repository on: GitHub

```
website-demo
```

Both you and your colleague are collaborators.

Push your local repository to GitHub.

Part A – Your PR

You create branch

```
git checkout -b feature-team-page
```

Create:

```
page7.html
```

Commit:

```
git add .  
git commit -m "Add Team page"
```

Push:

```
git push origin feature-team-page
```

Create PR

On GitHub:

```
feature-team-page  
↓  
main
```

Colleague Review

Ask them to:

- Add comment
- Request change

Example comment:

```
Please add team contact section.
```

Update PR

Add content.

Commit:

```
git add .  
git commit -m "Address review comments"  
  
git push
```

PR updates automatically.

Colleague Approves

They click:

```
Approve
```

Merge PR

Use:

Squash and Merge

(or regular merge)

Part B – Colleague's PR

Now reverse roles.

Colleague creates branch

feature-careers-page

Creates:

page8.html

Pushes and creates PR.

You review

Add comments:

Can you improve page heading?

Colleague updates

Pushes changes.

You approve

Merge PR.

Important GitHub PR Points

1. Reviewer vs Assignee

Reviewer → Person who reviews and approves/rejects the PR. 

Assignee → Person responsible for completing the work. Often the PR author, but many teams don't use it.

For your demo:

```
You create PR
↓
Colleague = Reviewer
```

That's enough.

2. After approval, who merges?

Common practices:

Small teams:

- PR author merges after approval.

Larger teams:

- Reviewer or tech lead may merge.

Most common workflow:

```
Author creates PR
↓
Reviewer approves
↓
Author merges
```

3. Standard way to resolve PR conflicts?

Recommended: Resolve on local machine.

Steps:

```
git checkout feature-branch
git fetch origin
git merge origin/main
# OR
git rebase origin/main
```

Resolve conflicts.

```
git add .
git commit      # if merge
# OR
git rebase --continue

git push
```

PR updates automatically.

GitHub web conflict resolution is generally used only for very simple text conflicts.

4. What is `git fetch`? How is it different from `git pull`?

`git fetch`

```
git fetch origin
```

- Downloads latest commits from remote.
- Updates remote-tracking branches like `origin/main`.
- Does **not** change your current branch or files.

Example:

Before fetch:

```
main      -> A
origin/main -> A
```

```
Remote has:
A---B
```

After fetch:

```
main      -> A
origin/main -> B
```

Your working copy is unchanged.

`git pull`

```
git pull
```


is roughly:

```
git fetch
git merge
```

It downloads changes **and immediately integrates them into your current branch.**

5. While resolving conflict on local machine we merge **main** into **feature**. Why?

Yes. This is very common when a PR has conflicts or the feature branch is behind main.

Example:

```
main
A---B---C

feature
A---B---D---E
```

Before merging feature into main, you first update feature:

```
git checkout feature
git fetch origin
git merge origin/main
```

Result:

```
A---B---C
 \       \
  D---E---M (feature)
```

Now feature already contains the latest main changes, conflicts are resolved, and the PR can merge cleanly.

An alternative is:

```
git checkout feature
git fetch origin
git rebase origin/main
```

Many teams prefer this because it keeps history linear.

So for PR conflict resolution, **merging or rebasing main into the feature branch is the normal approach**, not the other way around. The actual PR still merges **feature** → **main** afterward (after pushing changes).

6. How to make review compulsory (Branch Protection)?

In GitHub:

```
Repository
→ Settings
→ Branches
→ Add Branch Protection Rule
```

Select:

```
Branch: main
```

Enable:

- ☒ Require a pull request before merging
- ☒ Require approvals

Set:

```
Required approvals: 1
```

Optional:

- ☒ Dismiss stale approvals when new commits are pushed
- ☒ Require status checks

After this, nobody can merge without approval. This is the standard enterprise setup.

One Extra Tip

Before every merge/rebase demo, run:

```
git log --oneline --graph --decorate --all
```

You understand Git much faster when you can visually see the commit graph before and after each operation.